

МИНОБРНАУКИ РОССИИ
САНКТ-ПЕТЕРБУРГСКИЙ ГОСУДАРСТВЕННЫЙ
ЭЛЕКТРОТЕХНИЧЕСКИЙ УНИВЕРСИТЕТ
«ЛЭТИ» ИМ. В.И. УЛЬЯНОВА (ЛЕНИНА)
Кафедра САПР

ОТЧЕТ

по лабораторной работе №6

по дисциплине «Компьютерная графика»

**ТЕМА: «Формирования реалистических изображений с использованием
простых моделей освещения одним или двумя точечными
источниками.»**

Студенты гр. 3311

Аршин А.Д.

Баймухамедов

Р.Р.

Пасечный Л.В.

Преподаватель

Колев Г. Ю.

Санкт-Петербург

2025

Цель работы:

Целью данной работы является разработка программного обеспечения для визуализации трехмерных сцен с реалистичным освещением и тенями, включая реализацию модели освещения Фонга и алгоритма проекции теней от точечного источника света.

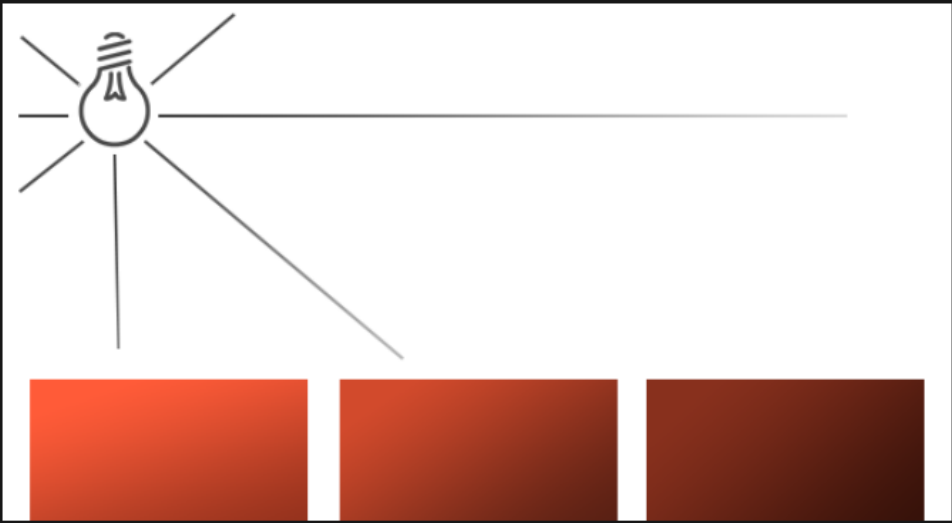
Задание (вариант 3):

Сформировать тени при освещении многоугольников и поверхностей, сформированных при выполнении темы 5, точечным источником освещения без учета интенсивности освещения тел, участвующих в сцене (с учетом зеркальной и диффузионной составляющих освещения). Обеспечить преобразование сцены при изменении координат источников освещения или наблюдателя

Теоретическая основа:

Точечные источники

Направленные источники света хорошо справляются с задачей освещения всей сцены целиком, но, обычно, нам требуется разместить дополнительно еще несколько точечных источников в пространстве сцены. Точечный источник света имеет заданное положение в пространстве и равномерно излучает во всех направлениях, а интенсивность освещения спадает с расстоянием. Обычные лампы накаливания или факелы можно привести как пример точечных источников.



В предыдущих уроках мы использовали простейшую реализацию точечного источника с заданной позицией, распространявшего свет во всех направлениях. Однако, лучи этого источника не затухали с удалением, что заставляло источник выглядеть чрезвычайно мощным. В большинстве же создаваемых сцен желательно было бы иметь источники, влияющие на окружение в непосредственной близости к ним, но не на всю сцену сразу.

Если вы добавите 10 контейнеров к сцене предыдущего примера, то заметите, что даже самые удаленные из них освещены с той же интенсивностью, что и находящиеся прямо перед источником. И это ожидаемо – мы ведь не ввели в расчет никаких выражений, управляющих интенсивностью освещения. Но мы хотим, чтобы удаленные контейнеры были лишь слегка подсвечены по сравнению с ближайшими к источнику.

Рис. 1

Затухание

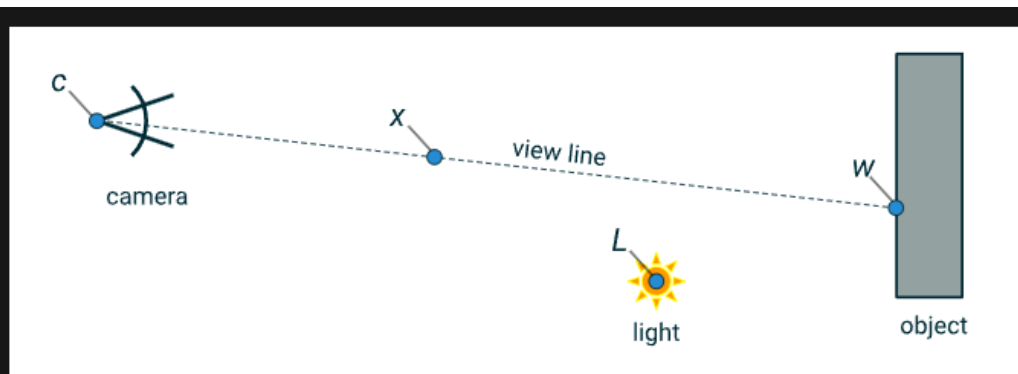
Ослабление интенсивности освещения с расстоянием называется затуханием. Простейшим методом расчета затухания является использование линейного закона. В этом случае интенсивность будет линейно спадать с расстоянием, обеспечивая меньшую освещенность удаленных объектов. Но визуально такая модель дает нереалистичные результаты. В общем случае реальные источники света дают сильную засветку вблизи, но интенсивность довольно резко падает в небольшом радиусе и далее оставшийся ее запас медленно уменьшается с расстоянием. Для имитации подобного поведения понадобится расчет похитрее линейного. К счастью, физики не дремлют, и обобщенное выражение вычисления коэффициента затухания от расстояния давно выведена. Остается только рассчитать значение для каждого фрагмента и умножить на результат вектор интенсивности конкретного источника:

$$F_{att} = \frac{1.0}{K_c + K_l d + K_q d^2}$$

, где d — расстояние от фрагмента до источника, K_c , K_l , K_q — постоянный, линейный и квадратичный коэффициенты.

- Постоянный коэффициент по обыкновению равен 1.0, выполняя защиту от уменьшения значения знаменателя до величин меньше единицы, что привело бы к росту интенсивности освещения на определенных расстояниях. Для нас такой эффект нежелателен.
- Линейный коэффициент определяет член, линейно уменьшающий интенсивность света с ростом расстояния.
- Квадратичный коэффициент определяет член, задающий квадратичный спад интенсивности. На малых расстояниях вклад квадратичного члена будет перекрыт вкладом линейного, но с ростом расстояния станет доминирующим в выражении.

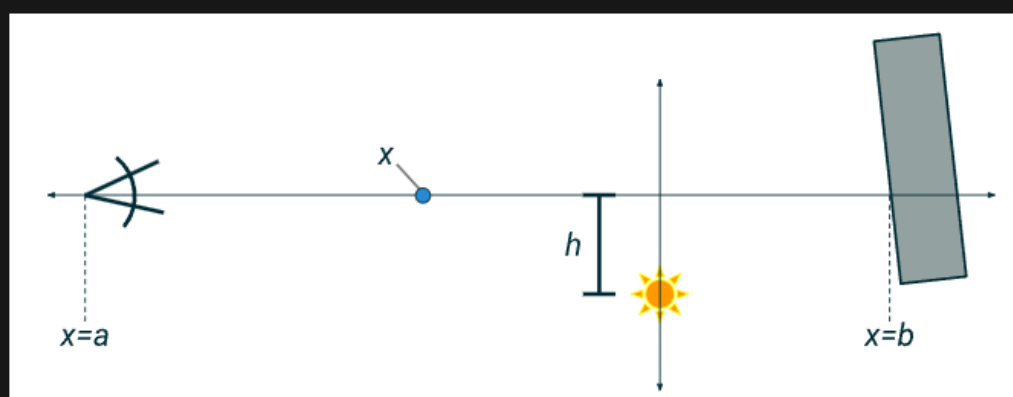
Рис. 2



Этот интеграл выражает именно то, что нам нужно, но его вычисление сложнее, чем нам требуется. Так как все переменные на показанном выше изображении являются векторами с тремя значениями, нам, по сути, придётся иметь дело с 12 переменными. Я устраню большинство этих переменных, выполнив простую репараметризацию.

Зададим новое пространство под названием «пространство освещения» (*light space*)

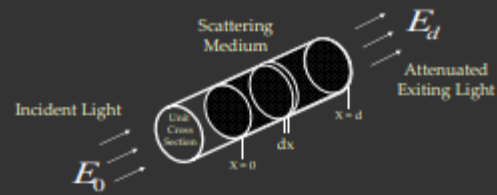
- По оси X откладывается линия взгляда
- По оси Y откладывается освещение



Теперь вместо линейного интеграла в трёхмерном пространстве я просто выполняю интегрирование по оси X из камеры до фрагмента мира. Далее нам нужно переписать интеграл. Расстояние от точки x на оси X до источника освещения равно $\sqrt{h^2 + x^2}$. Поэтому интеграл по линии взгляда получает вид

Рис. 3

Attenuation Model – Zeroth Order Scattering



Brightness at Distance d :

$$E(d) = E_0 e^{-\beta d} \quad (\text{Bouguer's Law, 1729})$$

Scattering Coefficient

Direct Transmission

Attenuation of Diverging Beams:

$$E(d) = \frac{I_0 e^{-\beta d}}{d^2} \quad (\text{Allard's Law, 1876})$$

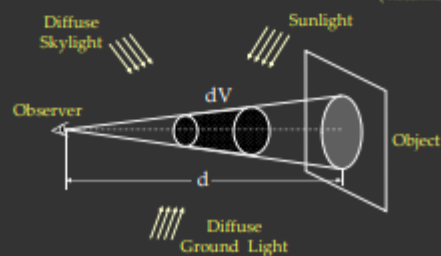
Optical Thickness:

$$T = \beta d$$

Scaled Depth

Airlight Model – First Order Scattering

(Koschmeider, 1924)



Brightness due to a Path of Length d :

$$E(d) = E_\infty (1 - e^{-\beta d})$$

Horizon Brightness

Рис. 4

Выполнение лабораторной работы

В качестве языка программирования для выполнения данной лабораторной работы выберем Python. Для быстрого запуска достаточно иметь установленный Python 3.

Демонстрация (<https://disk.yandex.ru/i/pVodYdxsuMOzsg>)

Интерфейс созданного приложения:

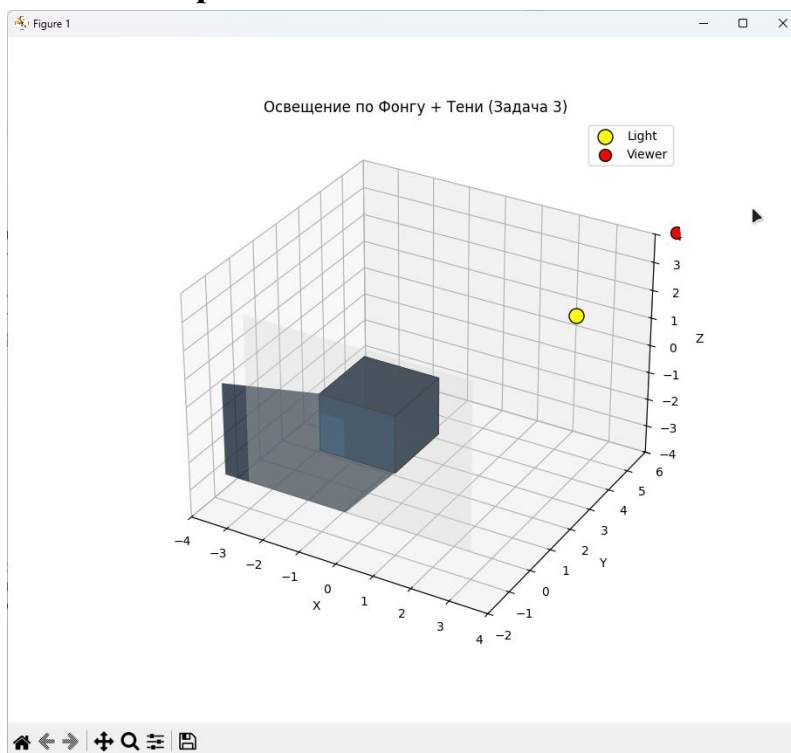


Рис. 3 Готовое приложение

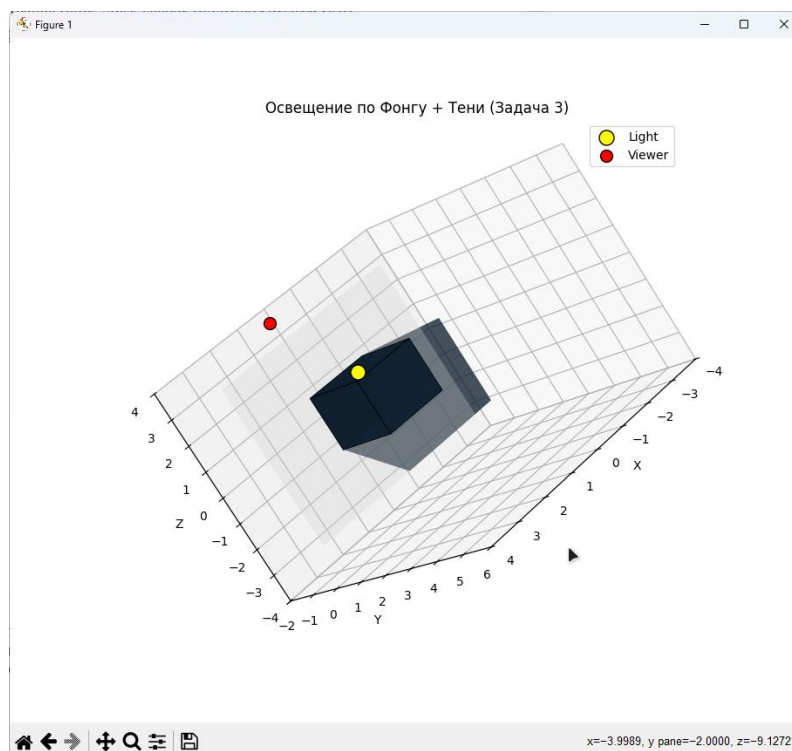


Рис. 4 Показ работы поворота в пространстве

Вывод:

В ходе выполнения работы была успешно создана система визуализации трехмерных сцен с реалистичным освещением и тенями. Анализ результатов позволяет сделать следующие выводы:

1. **Эффективность модели Фонга:** Реализованная модель освещения Фонга доказала свою адекватность для создания реалистичного визуального восприятия. Комбинация трех компонент (амбиентной, диффузной, зеркальной) позволяет точно передавать взаимодействие света с поверхностями объектов.
2. **Корректность теневой проекции:** Алгоритм проекции теней, основанный на параметрическом уравнении луча, обеспечивает точное определение теневых полигонов. Тени корректно отображаются на плоскости пола и соответствуют положению источника света.
3. **Качественная визуализация:** Система обеспечивает высококачественное отображение сцены с четким разделением освещенных и теневых областей. Использование разных цветов для источника света (желтый) и наблюдателя (красный) улучшает понимание пространственной организации сцены.
4. **Физическая достоверность:** Вычисление освещения для каждой грани отдельно с учетом ее нормали и положения обеспечивает физически корректное отображение. Особенно заметен эффект на ребрах куба, где яркость плавно изменяется в зависимости от ориентации граней.

Приложение 1: Листинг программы

```
import numpy as np
import matplotlib.pyplot as plt
# Позволяет создавать 3-д графику
from mpl_toolkits.mplot3d import Axes3D
# специальный объект для отрисовки 3-д полигонов
from mpl_toolkits.mplot3d.art3d import Poly3DCollection

# ----- Параметры освещения -----
light_pos = np.array([3.0, 4.0, 2.0]) # положение источника света
view_pos = np.array([5.0, 5.0, 5.0]) # положение наблюдателя

# Цвета
light_color = np.array([1.0, 1.0, 1.0]) # белый свет
object_color = np.array([0.2, 0.5, 0.8]) # базовый цвет объекта (RGB)

# Коэффициенты Фонга
ka = 0.2 # AMBIENT
kd = 0.6 # DIFFUSE
ks = 0.4 # SPECULAR
```

```

shininess = 32 # насколько острый блик

# Плоскость пола
ground_y = -1.0

# ----- Вспомогательные функции -----

# нормализация вектора
# превращает любой вектор в единичный
# нужно для корректных вычислений углов
def normalize(v):
    norm = np.linalg.norm(v)
    return v / norm if norm != 0 else v

# нормаль к грани
# берется три вершины
# строит два вектора по ребрам
# и векторное произведение дает нормаль к плоскости
# и в конечном итоге нормализует ее
def compute_face_normal(v0, v1, v2):
    # Нормаль к грани (векторное произведение)
    edge1 = v1 - v0
    edge2 = v2 - v0
    normal = np.cross(edge1, edge2)
    return normalize(normal)

# модель освещения Фонга
# вычисляется итоговый цвет грани с учетом, фонового, рассеяного и зеркального
# освещения
def phong_lighting(face_center, normal, light_pos, view_pos, color):

    # вектор от точки к источнику света
    L = normalize(light_pos - face_center)
    # вектор от точки к глазу наблюдателя
    V = normalize(view_pos - face_center)
    # направление отражённого луча света
    R = normalize(2 * np.dot(normal, L) * normal - L)

    # фоновое освещение
    ambient = ka * color

    # Диффузное освещение по закону Ламберта,
    # зависит от косинуса угла между нормалью и светом
    diff = max(np.dot(normal, L), 0) # чтобы небыло отрицательного света
    diffuse = kd * diff * color

    # зеркальный блик, яркость зависит от того, насколько отраженный луч попадает в
    # глаз
    # возводим в степень shininess, чтобы сделать блик узким
    spec = max(np.dot(R, V), 0) ** shininess
    specular = ks * spec * light_color

    # складываем все компоненты
    final_color = ambient + diffuse + specular

```



```

    return np.clip(final_color, 0, 1)

# точка тени
# это луч от источника света чеерез точку объекта
# Находим, где он пересекает плоскость пола (Y = ground_y).
# Параметрическое уравнение луча:  $P(t) = \text{light\_pos} + t * (\text{point} - \text{light\_pos})$ 
# Решаем уравнение:  $P(t).y = \text{ground\_y}$ 
# Если  $t < 0$  — пересечение позади источника → тени нет.
def project_shadow_point(point, light_pos, ground_y=0):
    direction = point - light_pos
    if direction[1] == 0:
        return None
    t = (ground_y - light_pos[1]) / direction[1]
    if t < 0:
        return None
    return light_pos + t * direction

# тень для всей грани
# проецирует каждую вершину грани на пол
# если хотя-бы одна не проецируется, то тень не рисуем
def project_shadow_polygon(vertices, light_pos, ground_y=0):
    shadow = []
    for v in vertices:
        s = project_shadow_point(np.array(v), np.array(light_pos), ground_y)
        if s is not None:
            shadow.append(s)
        else:
            return None
    return np.array(shadow)

# ----- Создание куба -----
def create_cube(size=1.0, center=(0, 0, 0)):
    cx, cy, cz = center
    s = size / 2
    vertices = np.array([
        [cx - s, cy - s, cz - s],
        [cx + s, cy - s, cz - s],
        [cx + s, cy + s, cz - s],
        [cx - s, cy + s, cz - s],
        [cx - s, cy - s, cz + s],
        [cx + s, cy - s, cz + s],
        [cx + s, cy + s, cz + s],
        [cx - s, cy + s, cz + s],
    ])
    faces = [
        [0, 1, 2, 3], # задняя
        [4, 7, 6, 5], # передняя
        [0, 4, 5, 1], # нижняя
        [2, 6, 7, 3], # верхняя
        [0, 3, 7, 4], # левая
        [1, 5, 6, 2], # правая
    ]
    return vertices, faces

```

```

# Создаём куб размером 2, в центре координат.
cube_vertices, cube_faces = create_cube(size=2.0, center=(0, 0, 0))

# ----- Вычисление цветов граней и теней -----

# списки для хранения цветов граней и их теней
face_colors = []
shadow_polys = []

# для каждой грани находим:
# центр и нормаль
# вычисляем цвет по Фонгу
# проецируем грань на пол, т.е. получаем цвет
for face in cube_faces:
    verts = cube_vertices[face]
    # Центр грани
    center = np.mean(verts, axis=0)
    # Нормаль (берём первые 3 точки)
    normal = compute_face_normal(verts[0], verts[1], verts[2])
    # Освещение по Фонгу
    color = phong_lighting(center, normal, light_pos, view_pos, object_color)
    face_colors.append(color)

    # Тень этой грани на пол
    shadow = project_shadow_polygon(verts, light_pos, ground_y)
    if shadow is not None:
        shadow_polys.append(shadow)

# Цвет тени — только амбиентный
shadow_color = ka * object_color # без диффузного и зеркального

# ----- Визуализация -----

# создаем 3-д графику
fig = plt.figure(figsize=(12, 9))
ax = fig.add_subplot(111, projection='3d')

# Рисуем куб Poly3DCollection принимает список полигонов.
# Каждая грань — отдельный полигон с вычисленным цветом.
# edgecolors='k' — чёрные рёбра.
# alpha=0.9 — чуть прозрачный.
for i, face in enumerate(cube_faces):
    verts = [cube_vertices[face]]
    ax.add_collection3d(Poly3DCollection(
        verts, facecolors=face_colors[i], edgecolors='k', linewidths=0.5, alpha=0.9
    ))

# Рисуем тени на полу
for shadow in shadow_polys:
    if len(shadow) >= 3:
        ax.add_collection3d(Poly3DCollection(
            [shadow], facecolors=shadow_color, alpha=0.5
        ))

```

```
# Источник света и наблюдатель
# желтая точка - свет
# красная точка - наблюдатель
ax.scatter(*light_pos, color='yellow', s=150, label='Light', edgecolor='black')
ax.scatter(*view_pos, color='red', s=100, label='Viewer', edgecolor='black')

# Пол
floor = np.array([
    [-3, ground_y, -3],
    [3, ground_y, -3],
    [3, ground_y, 3],
    [-3, ground_y, 3]
])
ax.add_collection3d(Poly3DCollection([floor], facecolors='lightgray', alpha=0.3))

# Настройки
# устанавливаем границы, подписи, легенду, заголовок
ax.set_xlim([-4, 4])
ax.set_ylim([-2, 6])
ax.set_zlim([-4, 4])
ax.set_xlabel('X')
ax.set_ylabel('Y')
ax.set_zlabel('Z')
ax.legend()
ax.set_title('Освещение по Фонгу + Тени (Задача 3)')

plt.show()
```